Lecture 6: Data Acquisition - APIs

James Sears*
AFRE 891/991 FS 26
Michigan State University

*Parts of these slides related to APIs are adapted from **"Data Science for Economists"** by Grant McDermott.

Table of Contents

1. Prologue

2. APIs

3. Explicit API Endpoint, No Key

4. Explicit API Endpoint with Key

5. Hidden APIs

6. Considerate Web Scraping

Prologue

Client-Side

Last time, we learned workflows for scraping **server-side** and **client-side** websites.

- Static sites with **rvest**
- Dynamic or interactive sites with **selenider** and **rvest**

Today, we'll turn our attention to **APIs**.

Packages for Today

First, a few packages for today:

```
pacman::p_load(lubridate, tidyverse, jsonlite, httr2, usethis,
               fredr, XML, xml2, robotstxt, rvest)
```

Also recommended: **JSON Viewer**, a browser extension for Firefox and Chrome that renders JSON output nicely.

- Alternative: **JSONView** for both Firefox and Chrome

APIs

APIs

An **API (Application Programming Interface)** is a set of rules that carries out all this requesting, responding, and rendering. Some key concepts:

- **Server:** A powerful computer that runs an API.
- **Client:** A program that exchanges data with a server through an API.
- **Protocol:** The "etiquette" underlying how computers talk to each other (e.g. HTTP).
- **Methods:** The "verbs" that clients use to talk with a server.
  - The main one that we'll be using is **GET** (i.e. ask a server to **retrieve information**).
  - Other common methods are POST, PUT and DELETE.

APIs

An **API (Application Programming Interface)** is a set of rules that carries out all this requesting, responding, and rendering. Some key concepts:

- **Request:** What the client asks of the server (see Methods on last slide).
- **Response:** The server's response. This includes:
  - **Status Code** (e.g. "404" if not found, or "200" if successful).
  - **Header** (i.e. meta-information about the response).
  - **Body** (i.e. the actual contents that we're interested in).

API Endpoints

With web APIs, we can access information *directly* from the API database if we can specify the correct URL, known as an **API endpoint**.

- Similar to normal URLs, but the resulting page is... visually unappealing



API Endpoints

The reason the site looks like hot 🗑 is that its structured and formatted in a very particular way (that can be read directly into R!).

Usually these pages are formatted in one of two ways: **XML** or **JSON**

- We don't need too worry too much about the particular syntax of either

In general there are **three types** of API Endpoints:

1. **Explicit** API Endpoint, **no API Key**
2. **Explicit** API Endpoint, **API Key Required**
3. **Hidden** API Endpoints

Let's work through an example workflow for each. But first...

Some APIs that are free & legal

Some fun APIs:

- **TikTok (unofficial)**
- **Spotify**
- **YouTube**

This list of **Public APIs** has many more.

R API Packages

Many **R packages** have been built to make directly working with APIs easier[1]. Some examples:

- **tidycensus** for **American Community Survey microdata and other Census Bureau data**
- **fredr** for **more easily doing what we will do today**
- **Quandl** for **Nasdaq and other financial information**
- **rtweet** for **Twitter data**
- **rnoaa** for **NOAA weather data**
- **prism** for **PRISM gridded climate data** (we'll work with this later on)
- **TADA** for **Water Quality Portal Data (EPA, USGS)**
- **rnassqs** for USDA's **QuickStats API** (agricultural data)
- **osmdata** for **OpenStreetMap spatial data**

[1] Note that a data acquisition project for this class requires you to engage *directly* with the API. Just using one of these packages doesn't count!

Explicit API Endpoint, No Key

Explicit API Endpoint, No Key

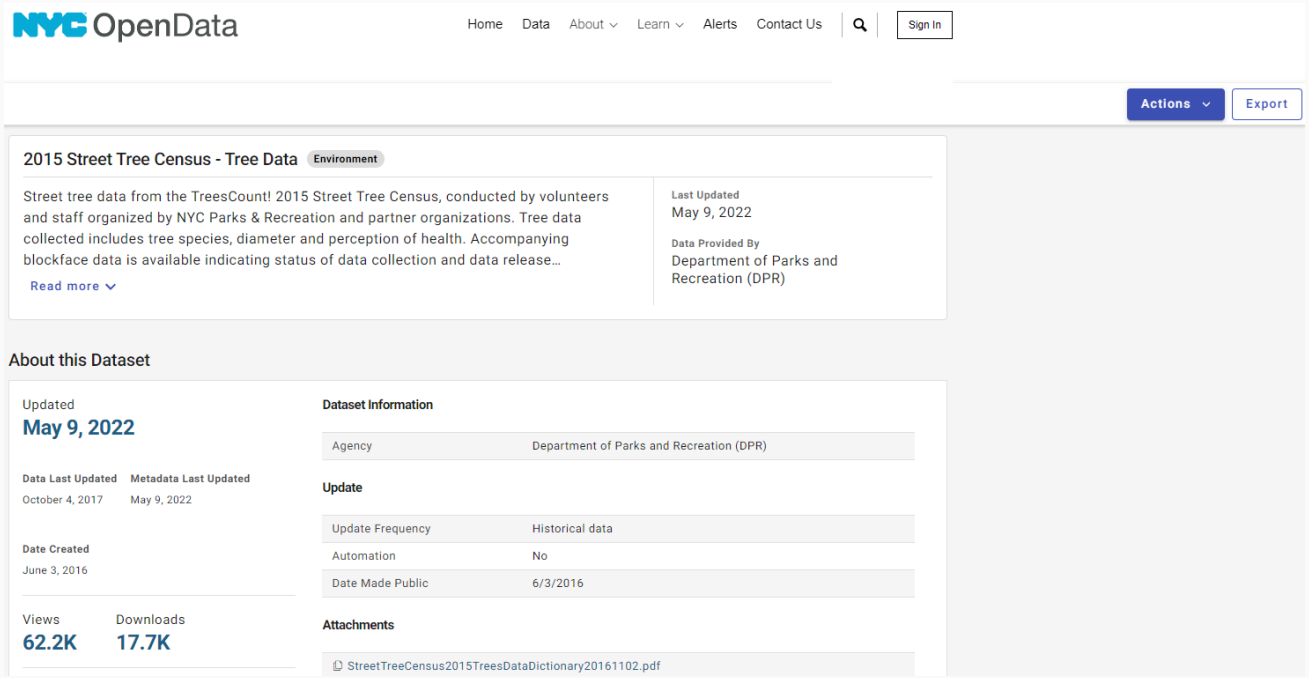
Let's work through an example of the first type of API: **explicit API without a key**.

Let's use the impressive **NYC Open Data** to look at... 🌲.

Seriously, trees.

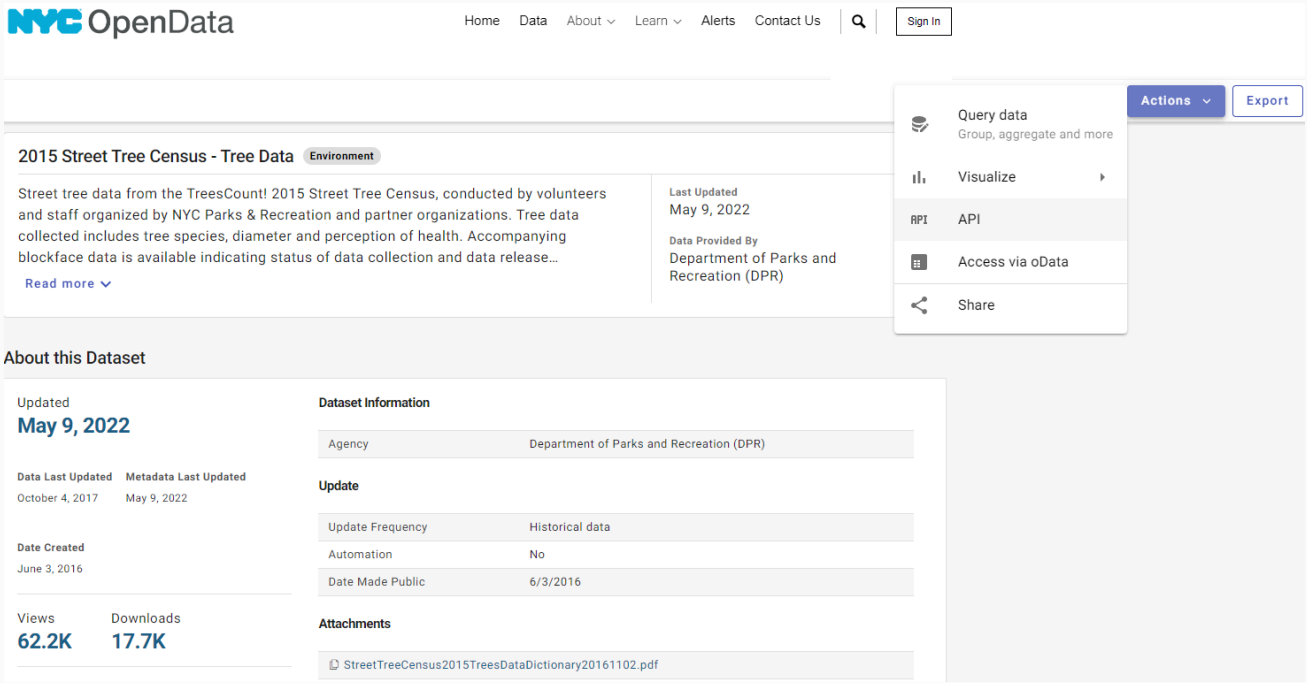
Application 1: NYC Tree Census

First, open the **Tree Census page** in your browser (if you haven't done so already)



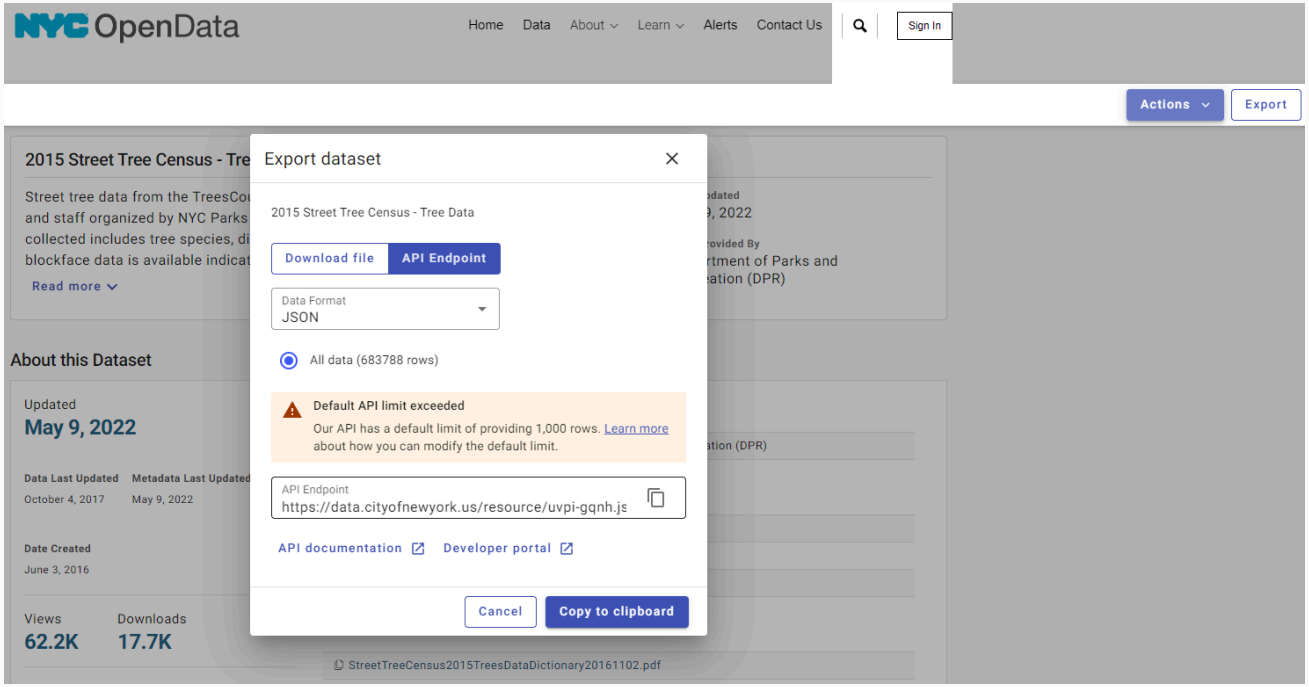
Application 1: NYC Tree Census

Click on **Actions > API** in upper-right to display the API Endpoint



Application 1: NYC Tree Census

Click on **Actions > API** in upper-right to display the API Endpoint



Application 1: NYC Tree Census

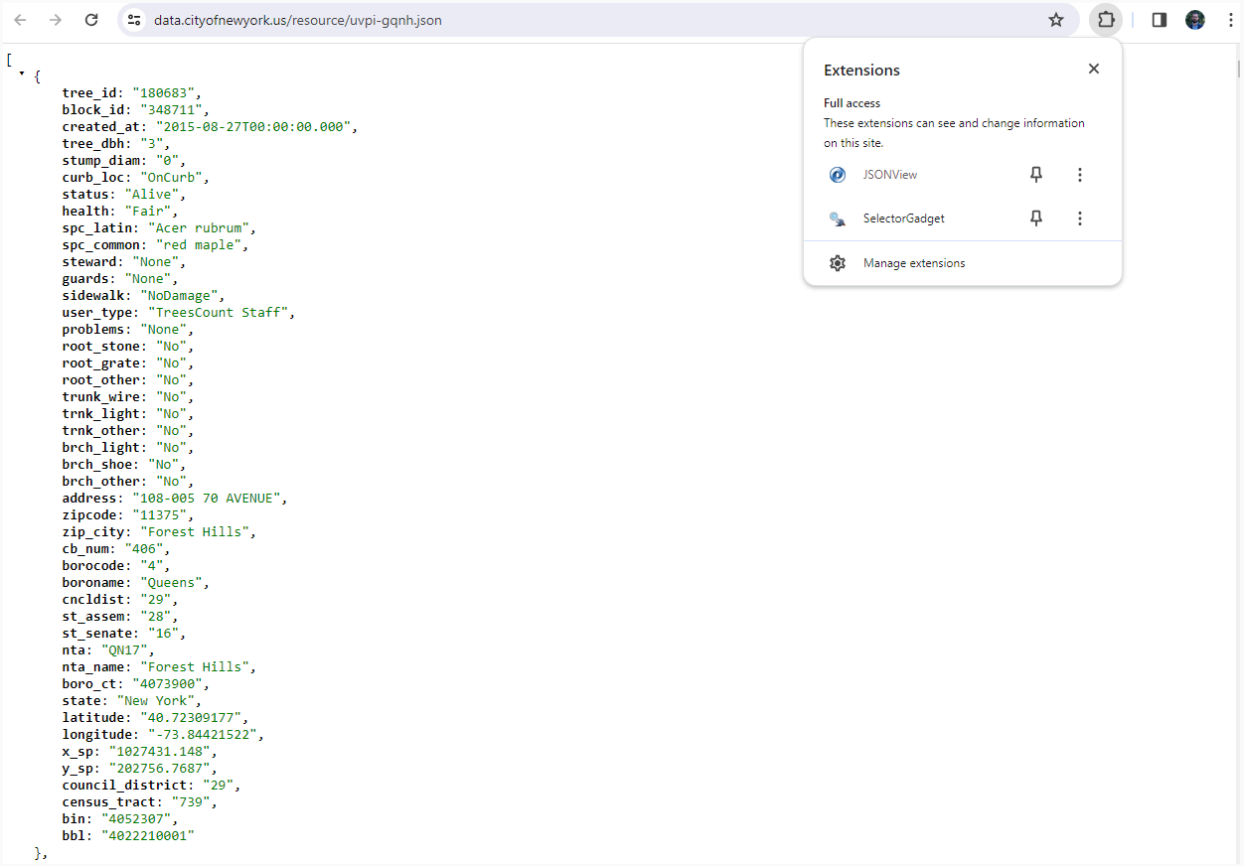
Copy/Paste or click "go to" to open the API endpoint link in another tab. You'll see a bunch of JSON text.



Application 1: NYC Tree Census

If you installed JSON Viewer/JSONView, it will look a lot nicer

- A lot like a transposed dataframe row...



Load JSON into R

Next, use the `fromJSON()` function in the **excellent jsonlite package** to read JSON into R.

```
nyc_trees ← fromJSON("https://data.cityofnewyork.us/resource/uvpi-gqnh.json") %>%
  as.data.frame()
nyc_trees
```

```
##       tree_id block_id              created_at tree_dbh stump_diam
## 1      180683   348711 2015-08-27T00:00:00.000        3          0
## 2      200540   315986 2015-09-03T00:00:00.000       21          0
## 3      204026   218365 2015-09-05T00:00:00.000        3          0
## 4      204337   217969 2015-09-05T00:00:00.000       10          0
## 5      189565   223043 2015-08-30T00:00:00.000       21          0
## 6      190422   106099 2015-08-30T00:00:00.000       11          0
## 7      190426   106099 2015-08-30T00:00:00.000       11          0
## 8      208649   103940 2015-09-07T00:00:00.000        9          0
## 9      209610   407443 2015-09-08T00:00:00.000        6          0
## 10     192755   207508 2015-08-31T00:00:00.000       21          0
## 11     203719   302371 2015-09-05T00:00:00.000       11          0
```

Aside: API Limits

Note that while the full census of NYC trees contains over 700,000 trees, we only downloaded a small portion.

By default, the API limits us to 1,000 rows.

For our use case (seeing an example of the workflow) this isn't an issue, but **reading the documentation** reveals a way to get around that (if so desired).

Punchline: always read the API documentation!

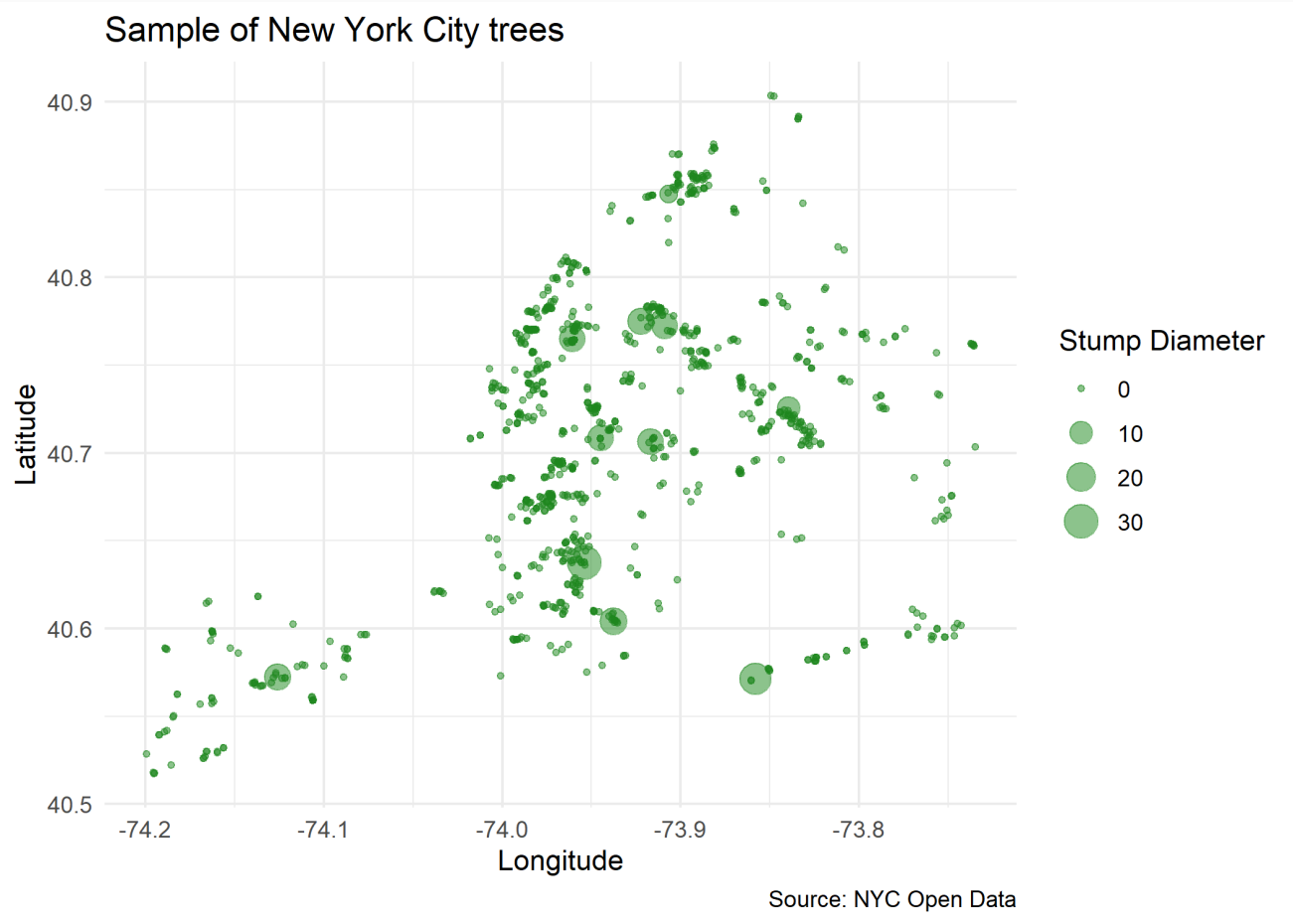
Work with the Data

Now we have the data, accessed through an API Endpoint!

Seriously, that's it. Now we can do whatever we want with the data - how about plotting it?

```
nyc_trees %>%
  select(longitude, latitude, stump_diam, spc_common, spc_latin, tree_id) %>%
  mutate_at(vars(longitude:stump_diam), as.numeric) %>%
  ggplot(aes(x=longitude, y=latitude, size=stump_diam)) +
  geom_point(alpha=0.5, color = "forestgreen") +
  scale_size_continuous(name = "Stump Diameter") +
  labs(
    x = "Longitude", y = "Latitude",
    title = "Sample of New York City trees",
    caption = "Source: NYC Open Data"
    ) +
  theme_minimal()
```

Plotting Some Trees



Explicit API Endpoint with Key

Explicit API Endpoint with Key

For the second application, we're going to focus on the more common circumstance: an API endpoint that **requires an API key**.

An **API key** is a way to add a **unique ID signature** to every API call you make

- Required for all APIs linked to gov't agencies or institutions (e.g. Census, BEA).

Let's do an application working with the **Federal Reserve Bank of St. Louis (FRED) API**

Application 2: FRED API

FRED makes available a **ton of U.S. economic data** that can easily be accessed via API.

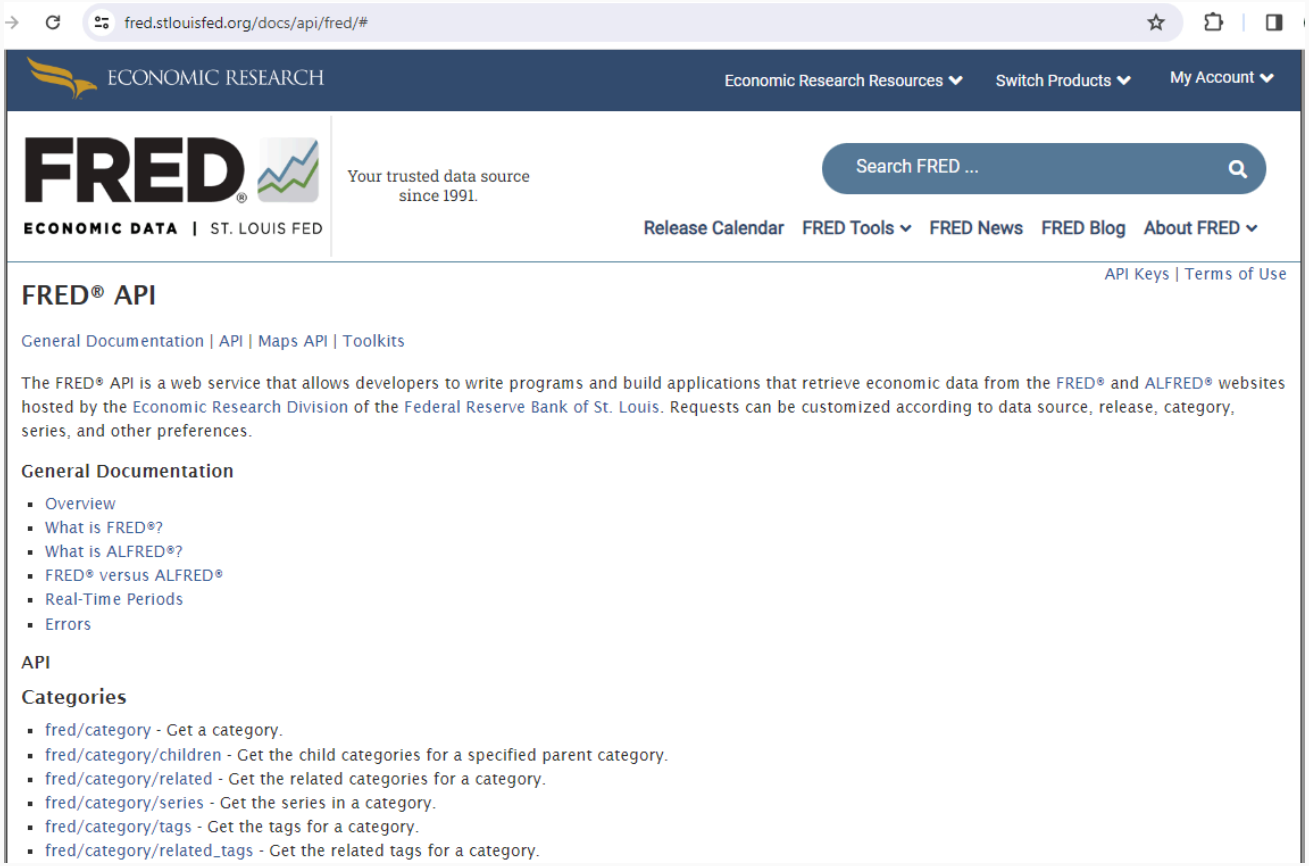


The only real difference from the last example: we first have to **register for an API key**

Application 2: FRED API

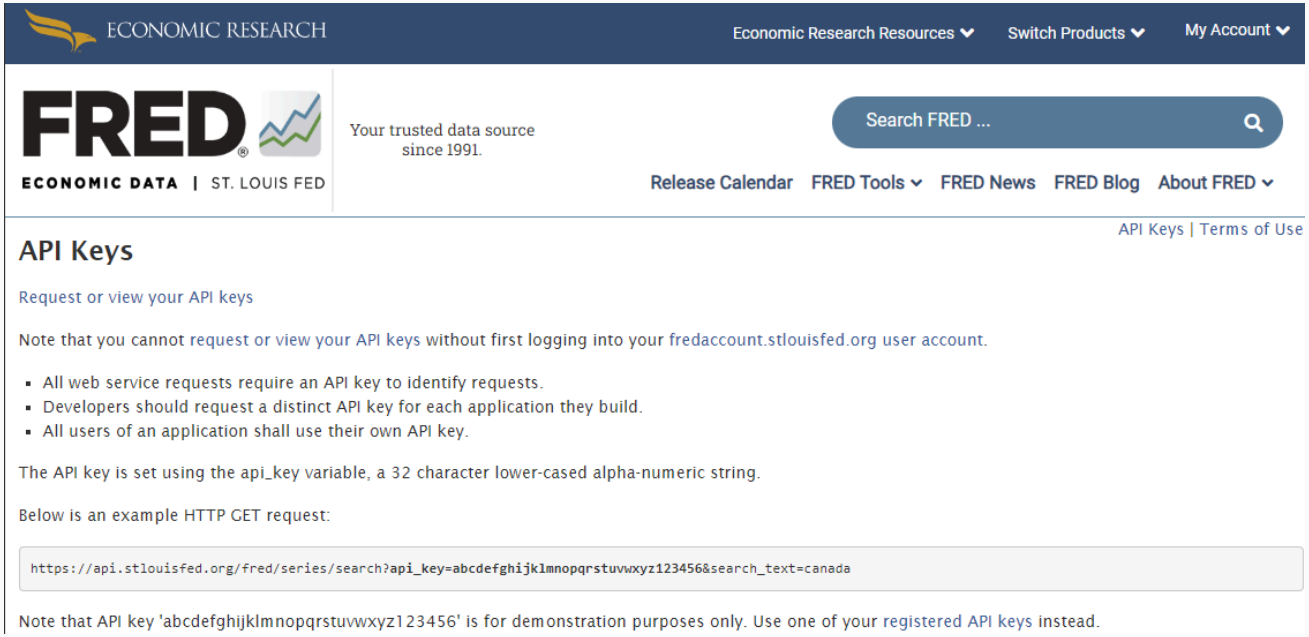
First, navigate to the **FRED API page**

```
knitr::include_graphics("images/fred_1.png")
```



Application 2: FRED API

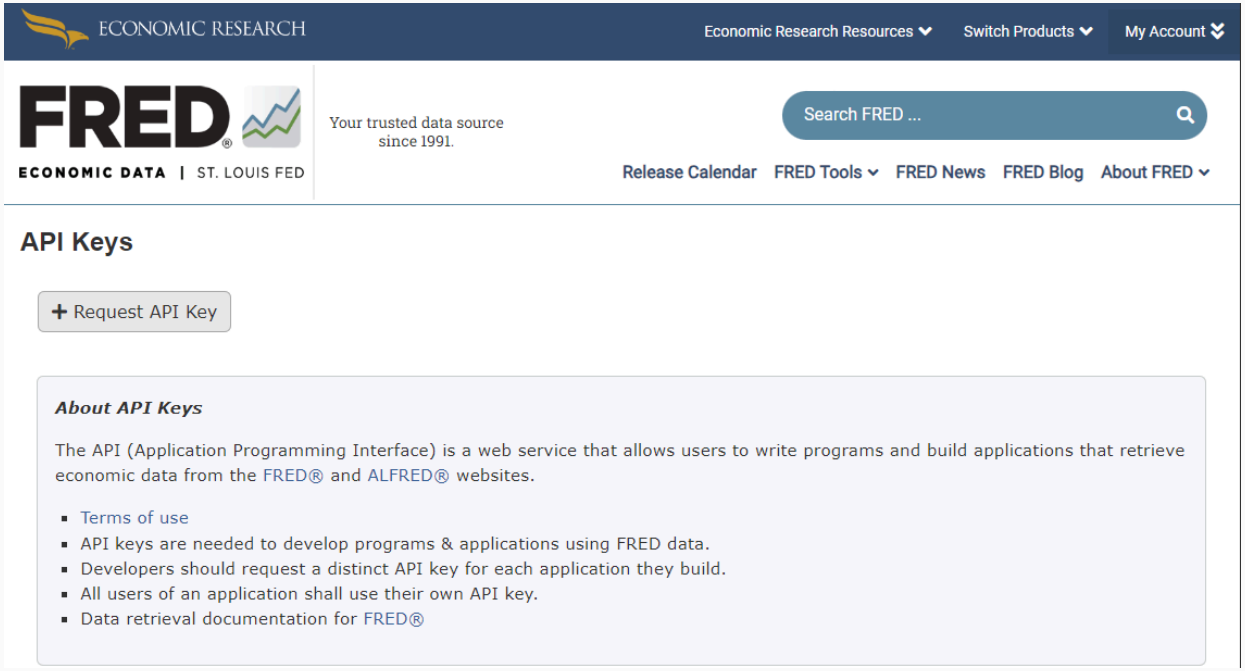
Click on "API Keys" in the upper-right then "Request or view your API keys"



Application 2: FRED API

Make an account, then request an API Key

- My Account > API Keys > Request API Key



Setting API Keys in R

Treat your API keys **like passwords**

- You're responsible for following API terms & conditions, and you can't control that if someone else is using your key

This means we shouldn't ever **hardcode API keys into R scripts**.

Instead, we can define them as **environment variables**, either

1. For the current R session only
2. Globally across all R sessions

Environment Variables: Current Session

To define an environment variable for **your current session only**, run the following **in your console:**[2]

```
Sys.setenv(FRED_API_KEY = "YOUR_KEY_HERE")
```

Replacing the fake API key with your actual key.

You can then use an environment variable in your script with

```
# assign to R object and print out
my_key ← Sys.getenv("FRED_API_KEY")
```

[2] setting `Sys.setenv()` in scripts defeats the entire purpose!

Environment Variables: Globally

To set an environment variable **globally across all sessions**, we'll need to add a special `~/.Renviron` file.

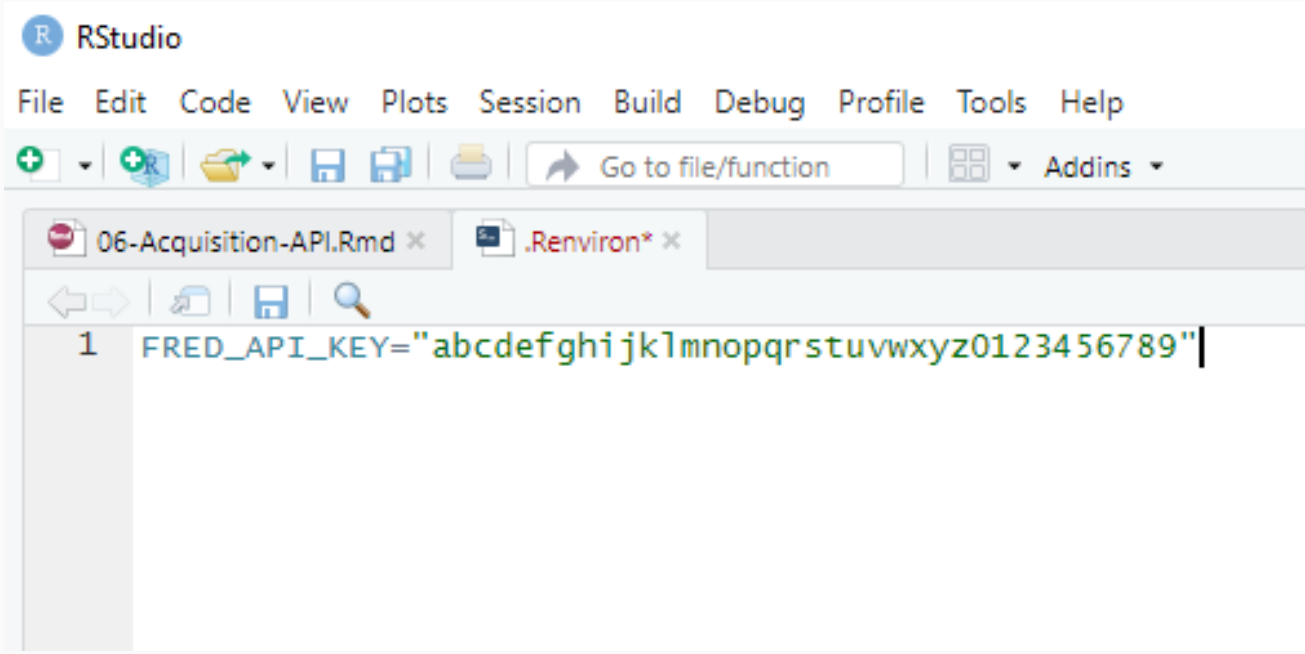
- Text file that lives in your home directory (hence the `~/` path)

To edit, we can use the `usethis::edit_r_environ()` function:

```
usethis::edit_r_environ()
```

Environment Variables: Globally

This opens your `~/.Renviron` file in a new tab, where you can easily add a line defining your API key.



Hit save, then restart R or refresh your environment session to use it:

```
## Only necessary when reading in a new Renviron variable
readRenviron("~/Renviron")
```

Using the FRED API

With your API key set, let's figure out how to use the FRED API.

Looking at the **FRED API documentation** you'll see

- There are many types of requests you can make of FRED using the API
- If we want the underlying data for a data series, we want the **fred/series/observations** path.

Using the FRED API

Clicking the link takes us to **specific documentation for that path**

Here we can learn

1. In which **file formats** the API will return data
    - XML, JSON, zipped tab delimited, zipped Excel
2. The exact **syntax** for making a request
    - Potentially specific to each format
3. The **parameters** that you can provide
    - e.g. function arguments

Using the FRED API

One required parameter is `series_id`.

- Let's look at the median home listing price in East Lansing.
- By searching the FRED website, I found that the **ID** for this series is `MELIPRMSA29620`.

For the `file_type`, you can use either XML or JSON.

- Both can easily be read into R.
- Since we already saw JSON, let's try using XML.

Try out the API right in your browser. Paste this into your location bar, replacing the fake API key with your own:

```
https://api.stlouisfed.org/fred/series/observations?
series_id=MELIPRMSA29620&api_key=YOURKEYHERE&file_type=json
```

Using the FRED API

To use the API in R, we'll use the tidyverse package **httr2** ("hitter 2"). This is an updated version of **httr** that adds more modern features and is pipeable!

- I find it much more intuitive

httr2 Request Worfklow

Recall that in **httr2**, we build + execute a request object over multiple function steps:

1. Create a new request object with `request()`
2. Define the request behavior with `req_` functions
3. Perform the request with `req_perform()`
4. Fetch the response with `resp_` functions

Request the FRED API

First we need to **create a request** with `request()`

```
req ← request("https://api.stlouisfed.org/")
```

httr2: Practice Request

```
req %>% req_dry_run()
```

```
## GET / HTTP/1.1
## accept: */*
## accept-encoding: deflate, gzip
## host: api.stlouisfed.org
## user-agent: httr2/1.3.0 r-curl/7.1.0 libcurl/8.14.1
```

Here we see that we've initiated a request of type **GET**. A **GET** method lets us "get" pre-existing data from a specific API endpoint

The alternative is a **POST** where we **submit data that results in creation of a new resource**

- **httr2** will automatically shift to **POST** if we add contents to our request!

httr2: Create Request

```
## GET / HTTP/1.1
## accept: */*
## accept-encoding: deflate, gzip
## host: api.stlouisfed.org
## user-agent: httr2/1.3.0 r-curl/7.1.0 libcurl/8.14.1
```

We also see 4 other useful bits of information that represent the **HTTP headers**

- We can modify these as needed later (i.e. the `user-agent` as before)

httr2: Modify Request

Once we have our request, we can **modify it** with the `req_` functions:

Function	Task
`req_body_(raw/file/json/json_modify/form/multipart`	send data in the request body (for a POST)
`req_(headers/headers_redacted`	modify the request headers
`req_(url/url_relative/url_query/url_path/url_path_append()`	modify the request URL
`req_progress()`	Add a progress bar (for long requests)
`req_timeout()`	Set time limit for a request
`req_user_agent()`	Sets the user-agent of the request
`req_proxy()`	Send the request through a proxy server
`req_cookie_(preserve/set)`	set and preserve cookies
`req_method()`	Set request to less common methods (HEAD, DELETE, PATCH, UPDATE, OPTIONS)

httr2: Modify Request

Let's start by telling the request the endpoint we want to query (`fred/series/observations`).

- Since we're appending to the FRED API url, we need `req_url_relative()`

```
req ← req_url_relative(req, "fred/series/observations")
req_dry_run(req)
```

```
## GET /fred/series/observations HTTP/1.1
## accept: */*
## accept-encoding: deflate, gzip
## host: api.stlouisfed.org
## user-agent: httr2/1.3.0 r-curl/7.1.0 libcurl/8.14.1
```

httr2: Modify Request

Now we need to add in some of our **query parameters**.

To do this, we need to follow the **syntax of the API documentation:**

```
?series_id=MELIPRMSA29620&api_key=YOURKEYHERE&file_type=json
```

Here we have **three parameters:**

- `series_id:` which series we want to get
- `api_key:` our personal API key
- `file_type:` the format we want the file in

httr2: Modify Request

We can add our **query parameters** using `req_url_query()`:

- Note: the parameter names are wrapped in **backticks**!
  - i.e. `` `series_id` = "MELIPRMSA29620" ``

```
req ← req_url_query(req,
           series_id = "MELIPRMSA29620",
           api_key = my_key,
           file_type = "xml")
req_dry_run(req)
```

```
## GET /fred/series/observations HTTP/1.1
## accept: */*
## accept-encoding: deflate, gzip
## host: api.stlouisfed.org
## user-agent: httr2/1.3.0 r-curl/7.1.0 libcurl/8.14.1
```

httr2: Make the Request

Once your request URL is set, use `req_perform()` to **perform the request**!

```
resp ← req_perform(req)
```

httr2: Looking at the Response

> GET https://api.stlouisfed.org/fred/series/observations?series_id=MELIPRMSA29620&api_key=YOURKEYHERE&file_type=xml
>
> Status: 200 OK
>
> Content-Type: text/xml
>
> Body: In memory (13176 bytes)

Looking at our response, we see that it worked (Status: 200 OK)!

httr2: Looking at the Response

What we want to do now is extract the body, which we can do with one of the `resp_body_` functions

Function	Task
`resp_body_raw`	return the raw bytes
`resp_body_string`	returns a UTF-8 string
`resp_body_json`	returns parsed JSON
`resp_body_html`	returns parsed HTML
`resp_body_xml`	returns parsed XML
`resp_has_body`	logical check if the response has a body

httr2: Looking at the Response

We requested XML, so let's use `resp_body_xml()` to parse it:

```
resp_body ← resp_body_xml(resp)
```

httr2: Looking at the Response

This gets us the XML document, which

- Starts with information about the data (start and end dates, order and sort, number of results, etc.)
- Contains each observation as a nested entry

```
resp_body
```

```
## {xml_document}
## <observations realtime_start="2026-10-14" realtime_end="2026-10-14"
observation_start="1600-01-01" observation_end="9999-12-31" units="lin"
output_type="1" file_type="xml" order_by="observation_date"
sort_order="asc" count="122" offset="0" limit="100000">
##  [1] <observation realtime_start="2026-10-14" realtime_end="2026-10-14"
date= ...
##  [2] <observation realtime_start="2026-10-14" realtime_end="2026-10-14"
date= ...
##  [3] <observation realtime_start="2026-10-14" realtime_end="2026-10-14"
date= ...
```

XML

See the XML structure with `xml_structure()`:

```
xml_structure(resp_body)
```

```
## <observations [realtime_start, realtime_end, observation_start,
observation_end, units, output_type, file_type, order_by, sort_order,
count, offset, limit]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
##   <observation [realtime_start, realtime_end, date, value]>
```

XML

This shows us the nested structure of our body, where each row is stored in a `<observation >` element.

We can retrieve these all using `xml_find_all()` and the xpath argument!

- Spoiler: it turns out that xpaths are actually the syntax for navigating an XML document

```
xml_find_all(resp_body, xpath = "//observation")
```

```
## {xml_nodeset (122)}
##  [1] <observation realtime_start="2026-10-14" realtime_end="2026-10-14"
date= ...
##  [2] <observation realtime_start="2026-10-14" realtime_end="2026-10-14"
date= ...
##  [3] <observation realtime_start="2026-10-14" realtime_end="2026-10-14"
date= ...
##  [4] <observation realtime_start="2026-10-14" realtime_end="2026-10-14"
date= ...
```

XML

> {xml_nodeset (111)}
>
> [1] <observation realtime_start="2026-10-13" realtime_end="2026-10-13" d ...
>
> [2] <observation realtime_start="2026-10-13" realtime_end="2026-10-13" d ...

`xml_find_all()` yields an xml nodeset, where each node contains four attributes: realtime_start, realtime_end, date, and value

- `xml_attrs()` to see the attributes

```
xml_find_all(resp_body, xpath = "//observation") %>%
xml_attrs()
```

```
## [[1]]
## realtime_start   realtime_end           date          value
##   "2026-10-14"   "2026-10-14"   "2016-07-01"            "."
##
## [[2]]
## realtime_start   realtime_end           date          value
```

XML

To extract a *specific* attribute's values, use `xml_attr("attribute_name")`

For example, to get the `realtime_start` values:

```
xml_find_all(resp_body, xpath = "//observation") %>%
xml_attr("realtime_start") %>% head()
```

```
## [1] "2026-10-14" "2026-10-14" "2026-10-14" "2026-10-14" "2026-10-14"
## [6] "2026-10-14"
```

XML

And we can build our dataframe!

```
resp_df ← data.frame(
  date = xml_find_all(resp_body, xpath = "//observation") %>%
xml_attr("date") %>% ymd(),
  median_price = xml_find_all(resp_body, xpath = "//observation") %>%
xml_attr("value") %>% as.numeric()
)
```

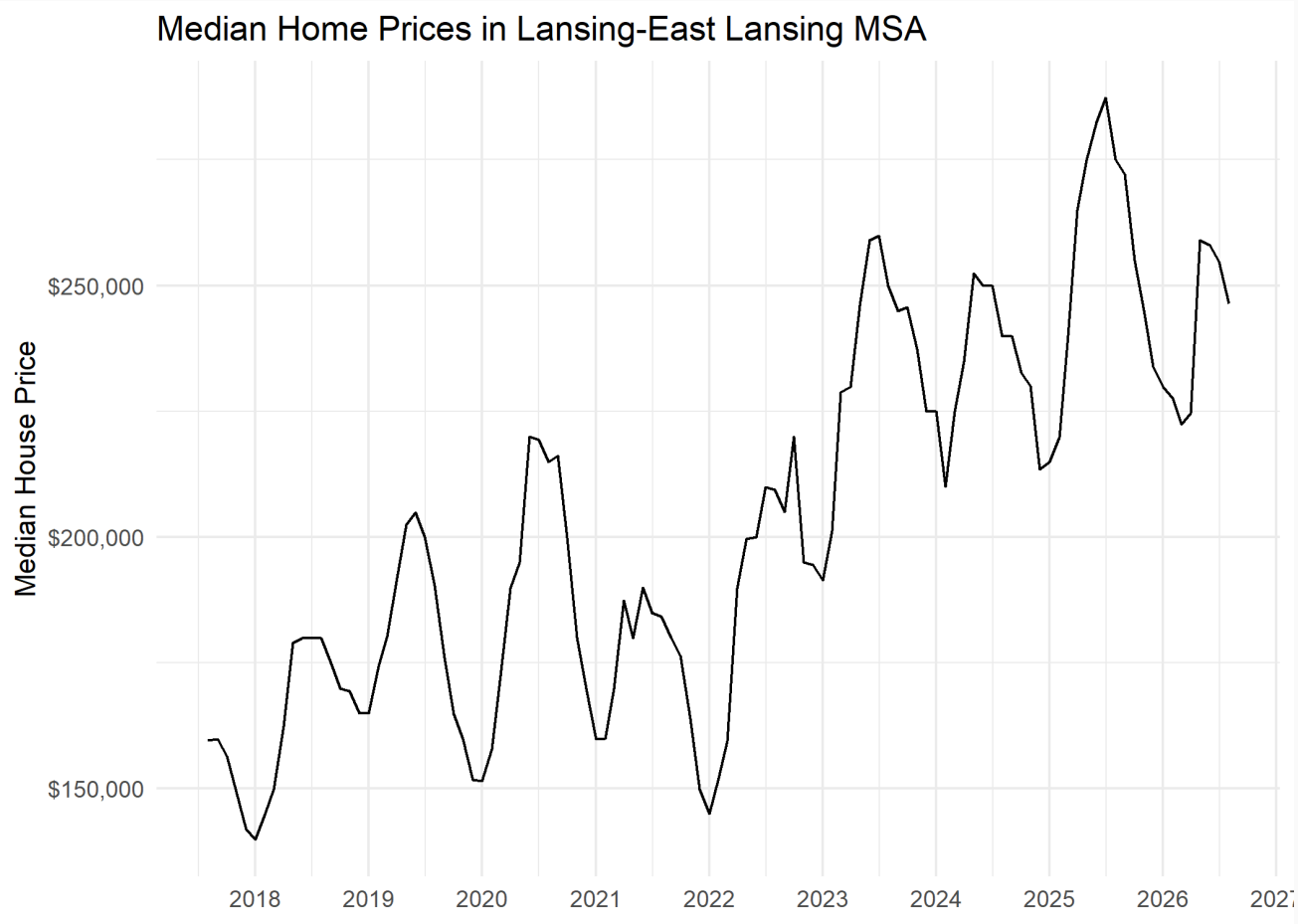
Using the FRED API

Plotting:

```
drop_na(resp_df) %>%
ggplot(aes(x = date, y = median_price)) +
  geom_line() +
  theme_minimal() +
  scale_y_continuous(labels = scales::label_dollar(),
                     name = "Median House Price") +
 scale_x_date(name = NULL,
               date_breaks = "1 year",
               date_labels = "%Y") +
  labs(title = "Median Home Prices in Lansing-East Lansing MSA")
```

Using the FRED API

Plotting:



Challenge

How has East Lansing's pandemic housing market compared with other cities?

- Get similar data from 2-3 other housing markets you're interested in.
- Plot time-series graphs for each housing market.
- More advanced: combine all the results in 1 dataframe (keep it tidy!).

Hidden APIs

Hidden APIs

The third kind of APIs are known as **Hidden APIs**.

These are endpoints used for a website's **Internal page generation** and is not intended for public use.

- For example, Airbnb uses or has used a hidden API to display search results on its web interface.
- Grant McDermott has an **example** of how to find and use hidden APIs.
- We're not going to go in-depth into hidden API access because...

Hidden APIs

One caveat: the legality of using hidden APIs is **less clear**

- In April 2022 (*HiQ Labs v. LinkedIn*), the Ninth Circuit Court of Appeals held that webscraping is legal in the U.S. so long as the data is **publicly available**
- Do hidden APIs constitute publicly available information?

Even if generally legal, web scraping may be a violation of a website's Terms & Conditions

- Airbnb: "Do not use bots, crawlers, scrapers, or other automated means to access or collect data or other content from or otherwise interact with the Airbnb Platform."
- Your IP could be blocked or your account could be blocked/blacklisted

Considerate Web Scraping

Considerate Web Scraping

In addition to our internal evaluation of

> Just because we can, doesn't mean we should

another useful tool in this process is **checking for and reading the robots.txt file**.

Many sites have a `robots.txt` file that outlines what parts of the site web crawlers *can* and *can't* access.

- Specific pages or API endpoints
- Specific types of users
- Speeds at which you can scrape

Considerate Web Scraping

The **robotstxt** package makes it easy to check for and read these files. For example, Wikipedia:

```
get_robotstxt(domain = "https://www.wikipedia.org")
```

```
## [robots.txt]
## --------------------------------------
##
## # robots.txt for http://www.wikipedia.org/ and friends
## #
## # Please note: There are a lot of pages on this site, and there are
## # some misbehaved spiders out there that go _way_ too fast. If you're
## # irresponsible, your access to the site may be blocked.
## #
##
## # Observed spamming large amounts of https://en.wikipedia.org/?curid=NNNNNN
## # and ignoring 429 ratelimit responses, claims to respect robots:
## # http://mj12bot.com/
## User-agent: MJ12bot
## Disallow: /
##
## # advertising-related bots:
## User-agent: Mediapartners-Google*
## Disallow: /
##
## # Wikipedia work bots:
```

Robots.txt

For more than just the text, use the `robotstxt()` function instead:

```
rtxt ← robotstxt(domain = "wikipedia.org")
rtxt
```

```
## $domain
## [1] "wikipedia.org"
##
## $text
## [1] "# robots.txt for http://www.wikipedia.org/ and friends\n#\n# Please note: There are a lot
of pages on this site, and there are\n# some misbehaved spiders out there that go _way_ too fast.
If you're\n# irresponsible, your access to the site may be blocked.\n#\n\n# Observed spamming large
amounts of https://en.wikipedia.org/?curid=NNNNNN\n# and ignoring 429 ratelimit responses, claims
to respect robots:\n# http://mj12bot.com/\n\n[ ... 702 lines omitted ... ]"
##
## $robexclobj
## <Robots Exclusion Protocol Object>
## $bots
## [1] "MJ12bot"                    "Mediapartners-Google*"
## [3] "IsraBot"                    "Orthogaffe"
## [5] "UbiCrawler"                 "DOC"
## [7] ""                           "[ ...  29 items omitted ... ]"
##
## $comments
##   line                                                          comment
## 1    1               # robots.txt for http://www.wikipedia.org/ and friends
## 2    2                                                                  #
```

Robots.txt

We can check what's allowed, including

Page Permissions

```
rtxt$permissions
```

```
##        field
useragent
## 1   Disallow
MJ12bot
## 2   Disallow
Mediapartners-Google*
## 3   Disallow
IsraBot
## 4   Disallow
Orthogaffe
## 5   Disallow
UbiCrawler
## 6   Disallow
```

Crawl Delay

```
rtxt$crawl_delay
```

```
##         field  useragent value
## 1 Crawl-delay SemrushBot     5
```

Considerate Web Scraping

Another great R package is **polite**, which builds upon **rvest** to provide a workflow that both queries the `robots.txt` file and performs the scraping itself.

Two main functions/steps:

`bow()`

- Introduces the client to the host
- Asks permission to web scrape
  - Inquires against the `robots.txt`

`scrape()`

- Retrieves data from the remote server

polite

Let's see an example by getting **Des Linden's 2024 NYC Marathon Results**.



polite

Installing **polite** direct from GitHub:[3]

```
remotes::install_github("dmi3kno/polite")
pacman::p_load(polite)
```

[3] **polite** was recently delisted from CRAN for the **test site cheese.com being down**. It's back up so should be relisted soon, but for now we need to install it from GitHub.

polite

Start by introducing yourself with `bow()`:

```
bow("https://www.runnersworld.com/races-places/a62786311/des-linden-
11th-new-york-city-marathon-2024/", verbose = T)
```

```
## <polite session> https://www.runnersworld.com/races-
places/a62786311/des-linden-11th-new-york-city-marathon-2024/
##     User-agent: polite R package
##     robots.txt: 52 rules are defined for 6 bots
##    Crawl delay: 5 sec
##   The path is scrapable for this user-agent
```

polite

```
## <polite session> https://www.runnersworld.com/races-
places/a62786311/des-linden-11th-new-york-city-marathon-2024/
##     User-agent: polite R package
##     robots.txt: 52 rules are defined for 6 bots
##    Crawl delay: 5 sec
##   The path is scrapable for this user-agent
```

`bow()` creates a **polite session** and tells us what's allowed given the rules outlined in the `robots.txt`.

- Here, we start with a 5 sec delay.
  - To change the time, use `set_scrape_delay(time)`

polite

Now, let's try and scrape the results page using `scrape()`. Arguments:

- bow: the polite session
- query: anything you want to append to the url
- accept: what type of data to expect (e.g. HTML, JSON, CSV)
- verbose: extra feedback

```
rw_session ← bow("https://www.runnersworld.com/races-
places/a62786311/des-linden-11th-new-york-city-marathon-2024/", verbose
= T)

rw_des_24 ← scrape(rw_session, accept = "html")
```

polite robots.txt

We can view the robots file directly from our session as the `$robotstxt` object:

```
rw_session$robotstxt
```

```
## $domain
## [1] "https://www.runnersworld.com"
##
## $text
## [1] "\nUser-agent: AmazonAdBot\nUser-agent: Google-InspectionTool\nDisallow:\n\nUser-
agent: ChatGPT-User\nUser-agent: GPTBot\nUser-agent: OAI-SearchBot\nDisallow:
/au/\nDisallow: /cn/\n\n[ ... 42 lines omitted ... ]"
##
## $robexclobj
## <Robots Exclusion Protocol Object>
## $bots
## [1] "AmazonAdBot"           "Google-InspectionTool" "ChatGPT-User"
## [4] "GPTBot"                "OAI-SearchBot"         "*"
##
## $comments
## [1] line    comment
## <0 rows> (or 0-length row.names)
##
## $permissions
```

polite

What did we get back?

```
rw_des_24
```

```
## {html_document}
## <html lang="en-US">
## [1] <head>\n<meta http-equiv="Content-Type" content="text/html;
charset=UTF-8 ...
## [2] <body>\n<template id="page-settings" data-nav-height="148">
</template><di ...
```

A: an HTML document, that we know how to interact with!

polite

Using **rvest** methods to retrieve the text:

```
pars ← html_elements(rw_des_24, ".article-body-content") %>%
  html_elements("p")
pars[[1]] %>%
  html_text2() %>%
  str_extract("[0-9]:[0-9]+:[0-9]+")
```

```
## [1] "2:29:32"
```

Considerate Web Scraping

Best Practices for considerate web scraping:

- Check the `robots.txt` file and scrape what's allowed
- If no `robots.txt` file exists, stick to
  1. Public information
  2. Scraped at a reasonable speed (i.e. human frequency)
- Scrape the data **once**, not **every time you run the script**
- Ask yourself: "would I feel comfortable having an undergrad RA scrape this?"

Table of Contents

1. Prologue
2. APIs
3. Explicit API Endpoint, No Key
4. Explicit API Endpoint with Key
5. Hidden APIs
6. Considerate Web Scraping